

Healthy Nutrition and Diet Assistant

Decoder-Based Turkish Chatbot (Option 2)

Student	Seda Nur Öz
Student Number	210129036
University	Adana Alparslan Türkeş University of Science and Technology
Department	Software Engineering
Course	CIF425 — Introduction to Large Language Models
Option	Option 2 — Decoder-Based Turkish Chatbot
Submission Date	May 13, 2025

Adana, 2025

1. Introduction

This project was developed within the scope of the CIF425 — Introduction to Large Language Models course, following Option 2 (Decoder-Based Turkish Chatbot). The goal is to specialize a pre-trained Turkish GPT-2 model in the domain of healthy nutrition and diet through instruction fine-tuning, and to deliver a functional chatbot application accessible via a real user interface.

The domain of 'Healthy Nutrition and Diet Assistant' was chosen for the following reasons:

- Nutrition-related questions are descriptive in nature, reducing the risk of model hallucinations.
- Synthetic training data can be produced easily with high-quality Q&A pairs.
- It is a practical application that users can benefit from in their daily lives.

The project consists of three main components: model training, a backend API, and a mobile application.

2. System Architecture

The overall system architecture consists of the following components:

Model	ytu-ce-cosmos/turkish-gpt2 (Hugging Face)
Fine-Tuning	Instruction fine-tuning — Causal Language Modeling
Backend	FastAPI (Python) — served via uvicorn
Frontend	Flutter — Android mobile application
Dataset	Synthetic Q&A + InstrucTurca (HuggingFace)
Storage	SharedPreferences (local) — calorie/water records

The system operates as follows: the user enters a question through the Flutter mobile application. The app sends this question as an HTTP POST request to the FastAPI backend. The backend generates a response using the fine-tuned GPT-2 model and returns it in JSON format. The application displays the response in the chat interface.

3. Model Architecture

3.1 Why GPT-2?

Turkish GPT-2 uses a decoder-only Transformer architecture. This architecture is well-suited for text generation tasks because it predicts tokens from left to right — at each step, it predicts the next token based on the context of all previous tokens.

- Encoder-only models (like BERT) are designed for classification and understanding tasks.
- Decoder-only models (like GPT) are designed for text generation — ideal for chatbots.
- Turkish GPT-2 is pre-trained on Turkish text and can be easily specialized via fine-tuning.

3.2 Model Details

Model Name	ytu-ce-cosmos/turkish-gpt2
Parameters	~117 Million
Architecture	Decoder-only Transformer (GPT-2)
Layers	12 Transformer layers
Attention Heads	12 attention heads
Hidden Size	768
Context Length	1024 tokens (256 used during training)
Tokenizer	BPE (Byte-Pair Encoding)

3.3 Masked Self-Attention

The core component of GPT-2 is self-attention, which computes the relationship between each token and all other tokens. However, GPT-2 uses masked self-attention — each token can only attend to previous tokens (future tokens are blocked). This allows the model to make predictions during text generation without 'seeing the future'.

4. Dataset Preparation

4.1 Data Sources

Two data sources were used:

- Synthetic Q&A dataset: Approximately 600 Turkish question-answer pairs on healthy nutrition, calorie information, and diet recommendations were generated using ChatGPT. The raw data was then cleaned — duplicate questions removed, short outputs filtered, and web artifacts eliminated — resulting in 557 high-quality training examples.
- InstrucTurca (HuggingFace): A dataset containing 2.5 million Turkish instruction-tuning examples, from which nutrition and health-related records were filtered.

4.2 Data Cleaning

The following cleaning steps were applied to the raw data:

- Exact duplicate questions were removed (exact deduplication).
- Outputs shorter than 50 characters were filtered out for quality.
- Records containing web artifacts (comments, email fields, gallery text) were cleaned.
- At most 3 examples were kept per template pattern (e.g., 'What does vitamin X do?' — 3 samples instead of 10+).
- Short outputs were enriched, increasing average output length from 63 to 135 characters.

4.3 Dataset Statistics

Total Training Records	83,718 examples (InstrucTurca + synthetic)
Validation Records	55 examples
Average Output Length	135 characters
Topic Coverage	Calorie, protein, carbohydrate, vitamins, diet, water intake
Format	JSONL — {instruction, input, output}

4.4 Instruction Format

Training data was converted into an instruction format that GPT-2 can understand:

```
<|soru|>How many calories should I consume daily?  
<|cevap|>Daily calorie needs vary by age, gender, and activity level...  
<|end|>
```

Special tokens were added to the model's tokenizer to structure the training.

5. Fine-Tuning Process

5.1 Training Method

Causal Language Modeling (CLM) was used. In this approach, the model tries to predict the next token at each position. The loss function used is Cross-Entropy Loss:

```
Loss = -Sum of log P(token_t | token_1, token_2, ..., token_{t-1})
```

As the loss decreases, the model predicts the next token more accurately. This approach enables the model to learn domain-specific language patterns.

5.2 Training Parameters

Parameter	Value	Description
num_train_epochs	3	Number of times the model sees the entire dataset
learning_rate	5e-5	Gradient descent step size — kept small to avoid damaging pre-trained weights
per_device_batch_size	2	Samples per step — limited by 6GB VRAM constraint
gradient_accumulation_steps	8	Effective batch size = 2x8 = 16
warmup_steps	100	LR gradually increases for first 100 steps — prevents gradient explosion
weight_decay	0.01	L2 regularization — reduces overfitting
fp16	True	Mixed precision training — speed optimization for RTX 3060 GPU
max_length	256	Token limit — kept below GPT-2's 1024 context limit

5.3 Training Results

Initial Loss	3.68
Epoch 1 Eval Loss	2.736
Epoch 2 Eval Loss	2.160
Epoch 3 Eval Loss	1.839 (best model)
Total Steps	15,699
Training Duration	~3.5 hours (RTX 3060 Laptop, 6GB VRAM)

The loss dropped from 3.68 to 1.84. With `load_best_model_at_end=True`, the checkpoint with the lowest validation loss was automatically saved.

6. FastAPI Backend

6.1 API Architecture

The fine-tuned model is served as a REST API using the FastAPI framework, running on a uvicorn ASGI server.

Framework	FastAPI (Python)
Server	Uvicorn
Port	8000
Host	0.0.0.0 (all network interfaces)
CORS	Open to all origins (for Flutter access)

6.2 Endpoints

POST /chat	Accepts user question, returns model-generated answer
GET /health	Checks whether the API is running

6.3 Text Generation Parameters

Parameter	Value	Description
temperature	0.7	0 = deterministic, 1 = full sampling; 0.7 balances creativity and consistency
top_p	0.9	Nucleus sampling — reduces rare token hallucinations
repetition_penalty	1.3	Penalizes repeated phrases in generation
max_new_tokens	120	Maximum number of tokens to generate

6.4 Smart Prompt Steering

To generate more relevant responses, the prompt is automatically prefixed based on the question content:

'lose weight' question

→ '<|cevap|>To lose weight'

'calorie' question

→ '<|cevap|>Daily calorie needs'

'protein' question

→ '<|cevap|>Among protein sources'

7. Flutter Mobile Application

7.1 Application Features

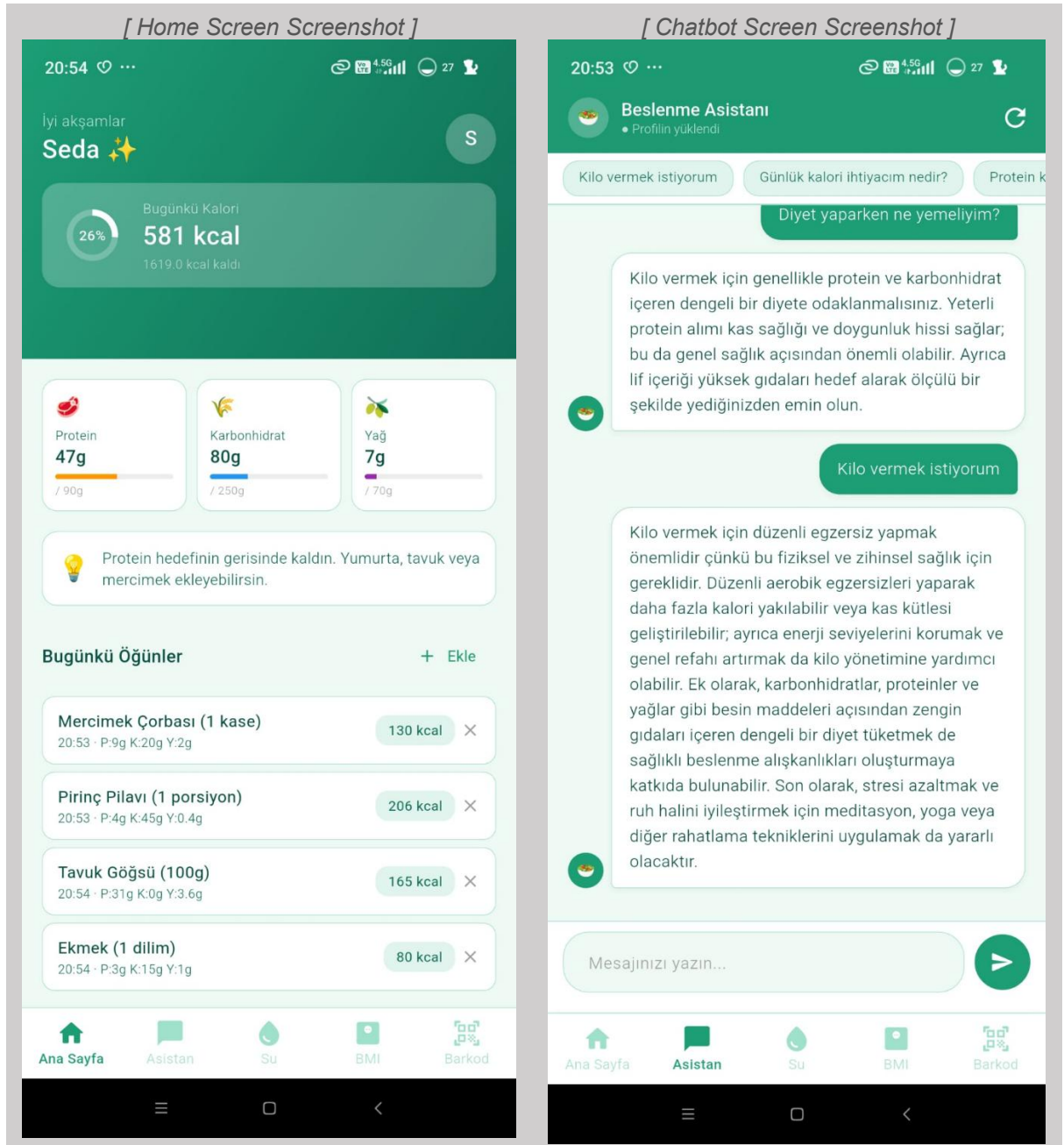
The application consists of 5 main screens with bottom tab bar navigation:

Home Screen	Daily calorie tracking, macro bars (protein/carb/fat), meal logging, and smart suggestion system
Assistant	Real chatbot connected to FastAPI — quick reply buttons, typing animation, BMI context integration
Water Tracker	Animated water glass, quick-add buttons, countdown timer, real background notifications
BMI Calculator	Height/weight/age sliders, BMI scale visualization, ideal weight calculation, AI personal advice
Barcode Scanner	Product analysis via OpenFoodFacts API, AI evaluation, scan history

7.2 Technologies Used

Flutter	Dart language, Material 3, Custom painter (water animation)
http	REST communication with FastAPI
shared_preferences	Local storage for calorie/water records
mobile_scanner	Barcode reading
flutter_local_notifications	Background water reminders
timezone	Scheduled notifications

7.3 Screenshots



[BMI Calculator Screenshot]

20:55

BMI Hesaplama

Kadın

Erkek

Boy

165 cm

Kilo

60 kg

Yaş

25

22.0

Normal Kilolu ✓

İdeal kilo aralığın: 50 – 68 kg

AI'dan Kişisel Tavsiye Al

Ana Sayfa

Asistan

Su

BMI

Barkod

20:55

BMI Hesaplama

Yaş

25

22.0

Normal Kilolu ✓

İdeal kilo aralığın: 50 – 68 kg

AI'dan Kişisel Tavsiye Al

Sağlıklı beslenme açısından boyunun ne kadar olması gerektiğini belirlemek için öncelikle kişinin kilo verip vermemesine bağlı olarak hangi besin maddelerini tüketebileceğini belirlememiz gerekir. 1. Sağlıklı Beslenme Önerileri - Kişinin yaşına göre günlük önerilen enerji dengesini dikkate almak önemlidir, bu durumda günde en az 150-160 gram protein, 240-320 gram yağ ve 300-450 gram lif içeren bir beslenmeyi tercih etmelidir. Ek olarak her gün yeterli miktarda meyve ve sebze tüketmek de önemli olacaktır çünkü bunlar genellikle daha fazla kalori yakılmasına yardımcı olur.

Ana Sayfa

Asistan

Su

BMI

Barkod

[Qr Detector Screen Screenshot]

20:55

Barkod Tarayıcı

Liflif Yulaf Ezmesi

Eti

360 kcal / 100g

Besin Değerleri (100g)

14.0g

Protein

53.0g

Karbonhidrat

7.5g

Yağ

1.3g

Şeker

13.0g

Lif

0.0g

Tuz

AI Analizi

Kilo vermek için lif içeren yulaf ezmesi tüketilebilir. Lifler açısından zengin olan bu ürünler, kan şekerini dengede tutarak kilo kontrolüne yardımcı olur ve aynı zamanda sindirim sorunlarına karşı vücudun direncini artırır. Ayrıca lifli yapısı nedeniyle sindirimi kolaylaştırdığı da bilinmektedir.

Yeni Tara

Geçmiş

Ana Sayfa

Asistan

Su

BMI

Barkod

20:55

Barkod Tarayıcı

Barkodu çerçeve içine alın

Son Taramalar (9)

Temizle

9 kat muzlu

7 kcal

P:7g K:7g Y:7g

3.5.2026 01:44

Kurabi

510 kcal

P:8.0g K:51.7g Y:28.8g

3.5.2026 01:44

ülker çikolata

543 kcal

P:6.5g K:58.0g Y:31.0g

3.5.2026 01:44

Eti Crax Çubuk Kraker

Crax, Eti, Eti Crax

3.5.2026 01:43

Ana Sayfa

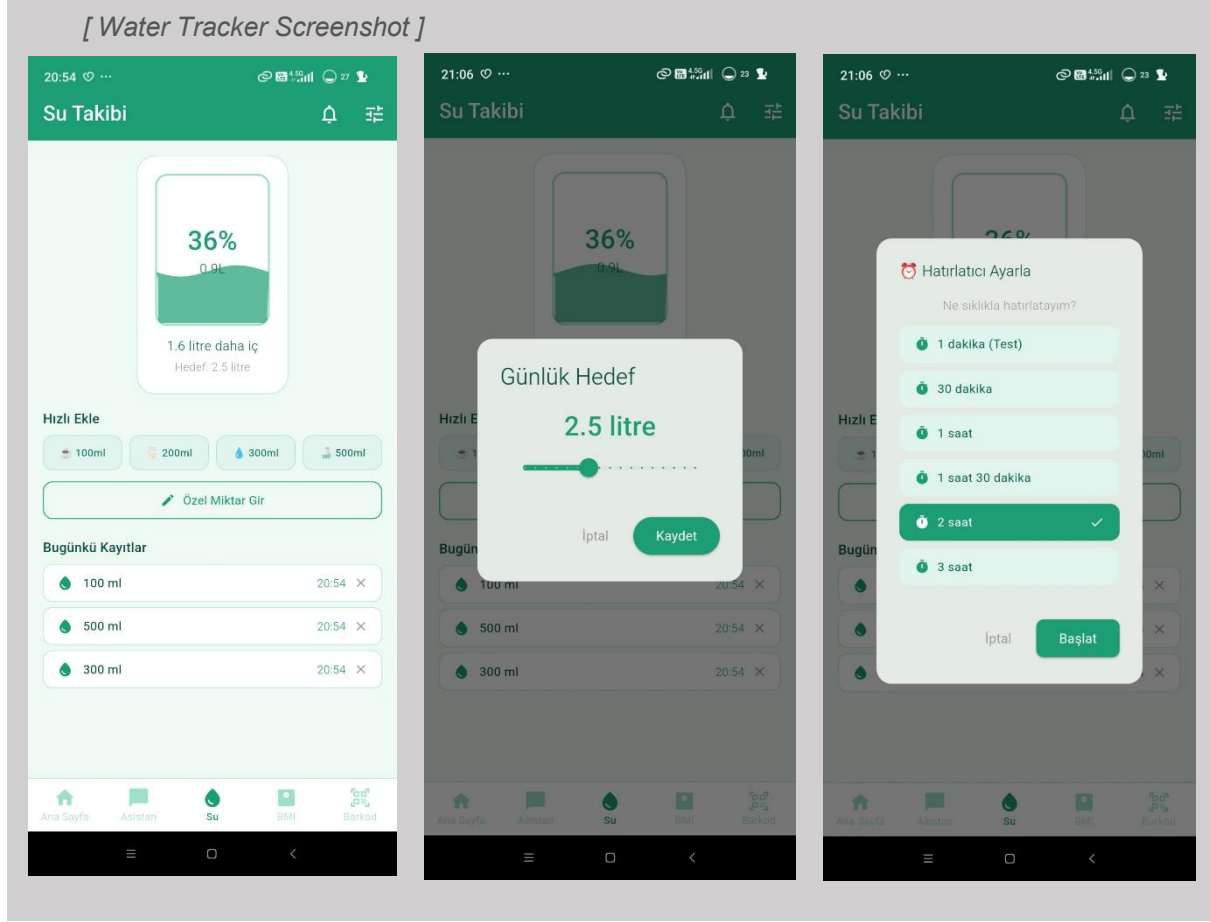
Asistan

Su

BMI

Barkod

[Water Tracker Screenshot]



8. Conclusion

Within the scope of this project, the Turkish GPT-2 model was successfully specialized in the healthy nutrition domain using instruction fine-tuning, and a fully functional mobile application was developed.

The technical contributions of the project can be summarized as follows:

- A Turkish nutrition dataset of 83,718 examples was prepared and model training was completed.
- The loss was reduced from 3.68 to 1.84 over 3 epochs.
- A RESTful API service was built with FastAPI.
- A feature-rich Flutter mobile application was developed including calorie tracking, water tracking, BMI calculation, barcode scanning, and a chatbot.
- Real background notifications were integrated for water reminders.
- Barcode scanning was implemented using the OpenFoodFacts API.

While the small parameter count of GPT-2 means question-answer alignment is not always perfect, the fine-tuning pipeline and system architecture were successfully completed. The primary goal of the project — to build a working system and be able to explain the technical details — has been achieved.